

FLOWER: A FRIENDLY FEDERATED LEARNING FRAMEWORK

Daniel J. Beutel^{1 2} Taner Topal^{1 2} Akhil Mathur³ Xinchu Qiu¹ Javier Fernandez-Marques⁴ Yan Gao¹
Lorenzo Sani⁵ Kwing Hei Li¹ Titouan Parcollet⁶ Pedro Porto Buarque de Gusmão¹ Nicholas D. Lane¹

ABSTRACT

Federated Learning (FL) has emerged as a promising technique for edge devices to collaboratively learn a shared prediction model, while keeping their training data on the device, thereby decoupling the ability to do machine learning from the need to store the data in the cloud. However, FL is difficult to implement realistically, both in terms of scale and systems heterogeneity. Although there are a number of research frameworks available to simulate FL algorithms, they do not support the study of scalable FL workloads on heterogeneous edge devices.

In this paper, we present Flower – a comprehensive FL framework that distinguishes itself from existing platforms by offering new facilities to execute large-scale FL experiments, and consider richly heterogeneous FL device scenarios. Our experiments show Flower can perform FL experiments up to *15M in client size* using only a pair of high-end GPUs. Researchers can then seamlessly migrate experiments to real devices to examine other parts of the design space. We believe Flower provides the community a critical new tool for FL study and development.

1 INTRODUCTION

There has been tremendous progress in enabling the execution of deep learning models on mobile and embedded devices to infer user contexts and behaviors (Fromm et al., 2018; Chowdhery et al., 2019; Malekzadeh et al., 2019; Lee et al., 2019; Yao et al., 2019; LiKamWa et al., 2016; Georgiev et al., 2017). This has been powered by the increasing computational abilities of mobile devices as well as novel algorithms which apply software optimizations to enable pre-trained cloud-scale models to run on resource-constrained devices. However, when it comes to the training of these mobile-focused models, a working assumption has been that the models will be trained centrally in the cloud, using training data aggregated from several users.

Federated Learning (FL) (McMahan et al., 2017) is an emerging area of research in the machine learning community which aims to enable distributed edge devices (or users) to collaboratively *train* a shared prediction model while keeping their personal data private. At a high level, this is achieved by repeating three basic steps: i) local parameters update to a shared prediction model on each edge device, ii) sending the local parameter updates to a central

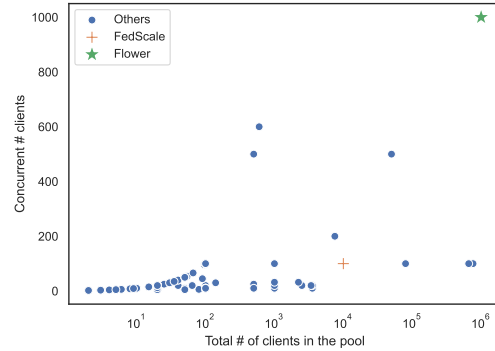


Figure 1. Survey of the number of FL clients used in FL research papers in the last two years. Scatter plot of number of concurrent clients participated in each communication round (y-axis) and total number of clients in the client pool (x-axis). The x-axis is converted to log scale to reflect the data points more clearly. FedScale can achieve 100 concurrent clients participated in each round out of 10000 total clients (orange point), while Flower framework can achieve 1000 concurrent clients out of a total 1 million clients (green point). The plot shows that Flower can achieve both higher concurrent participated client and larger client pool compared with other experiments existing the the recent research papers. Appendix A.1 gives details of the papers considered.

server for aggregation, and iii) receiving the aggregated model back for the next round of local updates.

From a systems perspective, a major bottleneck to FL research is the paucity of frameworks that support scalable execution of FL methods on mobile and edge devices. While several frameworks including Tensorflow Federated (Google, 2020; Abadi et al., 2016a) (TFF) and LEAF (Caldas et al., 2018) enable experimentation on FL

¹Department of Computer Science and Technology, University of Cambridge, UK ²Adap, Hamburg, Hamburg, Germany ³Nokia Bell Labs, Cambridge, UK ⁴Department of Computer Science, University of Oxford, UK ⁵Department of Physics and Astronomy, University of Bologna, Italy ⁶Laboratoire Informatique d'Avignon, Avignon Université, France. Correspondence to: Daniel J. Beutel <daniel@adap.com>.

algorithms, they do not provide support for running FL on edge devices. System-related factors such as heterogeneity in the software stack, compute capabilities, and network bandwidth, affect model synchronization and local training. In combination with the choice of the client selection and parameter aggregation algorithms, they can impact the accuracy and training time of models trained in a federated setting. The systems' complexity of FL and the lack of scalable open-source frameworks can lead to a disparity between FL research and production. While closed production-grade systems report client numbers in the thousands or even millions (Hard et al., 2019), few research papers use populations of more than 100 clients, as can be seen in Figure 1. Even those papers which use more than 100 clients rely on simulations (e.g., using nested loops) rather than actually implementing FL clients on real devices.

In this paper, we present *Flower*¹, a novel FL framework, that supports experimentation with both algorithmic and systems-related challenges in FL. Flower offers a stable, language and ML framework-agnostic implementation of the core components of a FL system, and provides higher-level abstractions to enable researchers to experiment and implement new ideas on top of a reliable stack. Moreover, Flower allows for rapid transition of existing ML training pipelines into a FL setup to evaluate their convergence properties and training time in a federated setting. Most importantly, Flower provides support for extending FL implementations to mobile and wireless clients, with heterogeneous compute, memory, and network resources.

As system-level challenges of limited compute, memory, and network bandwidth in mobile devices are not a major bottleneck for powerful cloud servers, Flower provides built-in tools to simulate many of these challenging conditions in a cloud environment and allows for a realistic evaluation of FL algorithms. Finally, Flower is designed with scalability in mind and enables large-cohort research that leverages both a large number of connected clients and a large number of clients training concurrently. We believe that the capability to perform FL at scale will unlock new research opportunities as results obtained in small-scale experiments are not guaranteed to generalize well to large-scale problems. In summary, we make the following contributions:

- We present Flower, a novel FL framework that supports large-cohort training and evaluation, both on real edge devices and on single-node or multi-node compute clusters. This unlocks scalable algorithmic research of real-world system conditions such as limited computational resources which are common for typical FL workloads.
- We describe the design principles and implementation details of Flower. In addition to being language- and

ML framework-agnostic by design, Flower is also fully extendable and can incorporate emerging algorithms, training strategies and communication protocols.

- Using Flower, we present experiments that explore both algorithmic and system-level aspects of FL on five machine learning workloads with up to 15 million clients. Our results quantify the impact of various system bottlenecks such as client heterogeneity and fluctuating network speeds on FL performance.
- *Flower is open-sourced under Apache 2.0 License* and adopted by major research organizations in both academia and industry. The community is actively participating in the development and contributes novel base-lines, functionality, and algorithms.

2 BACKGROUND AND RELATED WORK

FL builds on a vast body of prior work and has since been expanded in different directions. McMahan et al. (2017) introduced the basic federated averaging (FedAvg) algorithm and evaluated it in terms of communication efficiency. There is active work on privacy and robustness improvements for FL: A targeted model poisoning attack using Fashion-MNIST (Xiao et al., 2017) (along with possible mitigation strategies) was demonstrated by Bhagoji et al. (2018). Abadi et al. (2016b) propose an attempt to translate the idea of differential privacy to deep learning. Secure aggregation (Bonawitz et al., 2017) is a way to hide model updates from “honest but curious” attackers. Robustness and fault-tolerance improvements at the optimizer level are commonly studied and demonstrated, e.g., by Zeno (Xie et al., 2019). Finally, there is an increasing emphasis on the performance of federated optimization in heterogeneous data and system settings (Smith et al., 2017; Li et al., 2018; 2019).

The optimization of distributed training with and without federated concepts has been covered from many angles (Dean et al., 2012; Jia et al., 2018; Chahal et al., 2018; Sergeev & Balso, 2018; Dryden et al., 2016). Bonawitz et al. (2019) detail the system design of a large-scale Google-internal FL system. TFF (Google, 2020), PySyft (Ryffel et al., 2018), and LEAF (Caldas et al., 2018) propose open source frameworks which are primarily used for simulations that run a small number of *homogeneous* clients. Flower unifies both perspectives by being open source and suitable for exploratory research, with scalability to expand into settings involving a large number of *heterogeneous* clients. Most of the mentioned approaches have in common that they implement their own systems to obtain the described results. The main intention of Flower is to provide a framework which would (a) allow to perform similar research using a common framework and (b) enable to run those experiments on a large number of *heterogeneous* devices.

¹<https://flower.dev>

3 FLOWER OVERVIEW

Flower is a novel end-to-end federated learning framework that enables a more seamless transition from experimental research in simulation to system research on a large cohort of real edge devices. Flower offers individual strength in both areas (viz. simulation and real world devices); and offers the ability for experimental implementations to migrate between the two extremes as needed during exploration and development. In this section, we describe use cases that motivate our perspective, design goals, resulting framework architecture, and comparison to other frameworks.

3.1 Use Cases

The identified gap between FL research practice and industry reports from proprietary large-scale systems (Figure 1) is, at least in part, related a number of use cases that are not well-supported by the current FL ecosystem. The following sections show how Flower enables those use cases.

Scale experiments to large cohorts. Experiments need to scale to both a large client pool size and a large number of clients training concurrently to better understand how well methods generalize. A researcher needs to be able launch large-scale FL evaluations of their algorithms and design using reasonable levels of compute (e.g., single-machine/a multi-GPU rack), and have results at this scale have acceptable speed (wall-clock execution time).

Experiment on heterogeneous devices. Heterogeneous client environments are the norm for FL. Researchers need ways to both simulate heterogeneity and to execute FL on real edge devices to quantify the effects of system heterogeneity. Measurements about the performance of client performance should be able to be easily collected, and deploying heterogeneous experiments is painless.

Transition from simulation to real devices. New methods are often conceived in simulated environments. To understand their applicability to real-world scenarios, frameworks need to support seamless transition between simulation and on-device execution. Shifting from simulation to real devices, mixing simulated and real devices, and selecting certain elements to have varying levels of realism (e.g., compute or network) should be easy.

Multi-framework workloads. Diverse client environments naturally motivate the usage of different ML frameworks, so FL frameworks should be able to integrate updates coming from clients using varying ML frameworks in the same workload. Examples range from situations where clients use two different training frameworks (pytorch and tensorflow) to more complex situations where clients have their own device- and OS-specific training algorithm.

Table 1. Excerpt of built-in FL algorithms available in Flower. New algorithms can be implemented using the *Strategy* interface.

Strategy	Description
FedAvg	Vanilla Federated Averaging (McMahan et al., 2017)
Fault Tolerant FedAvg	A variant of FedAvg that can tolerate faulty client conditions such as client disconnections or laggards.
FedProx	Implementation of the algorithm proposed by Li et al. (2020) to extend FL to heterogenous network conditions.
QFedAvg	Implementation of the algorithm proposed by Li et al. (2019) to encourage fairness in FL.
FedOptim	A family of server-side optimizations that include FedAdagrad, FedYogi, and FedAdam as described in Reddi et al. (2021).

3.2 Design Goals

The given uses cases identify a gap in the existing FL ecosystem that results in research that does not necessarily reflect real-world FL scenarios. To adress the ecosystem gap, we defined a set of independent design goals for Flower:

Scalable: Given that real-world FL would encounter a large number of clients, Flower should scale to a large number of concurrent clients to foster research on a realistic scale.

Client-agnostic: Given the heterogeneous environment on mobile clients, Flower should be interoperable with different programming languages, operating systems, and hardware.

Communication-agnostic: Given the heterogeneous connectivity settings, Flower should allow different serialization and communication approaches.

Privacy-agnostic: Different FL settings (cross-devic, cross-silo) have different privacy requirements (secure aggregation, differential privacy). Flower should support common approaches whilst not be prescriptive about their usage.

Flexible: Given the rate of change in FL and the velocity of the general ML ecosystem, Flower should be flexible to enable both experimental research and adoption of recently proposed approaches with low engineering overhead.

A framework architecture with those properties will increase both realism and scale in FL research and provide a smooth transition from research in simulation to large-cohort research on real edge devices. The next section describes how the Flower framework architecture supports those goals.

3.3 Core Framework Architecture

FL can be described as an interplay between global and local computations. Global computations are executed on the server side and responsible for orchestrating the learning

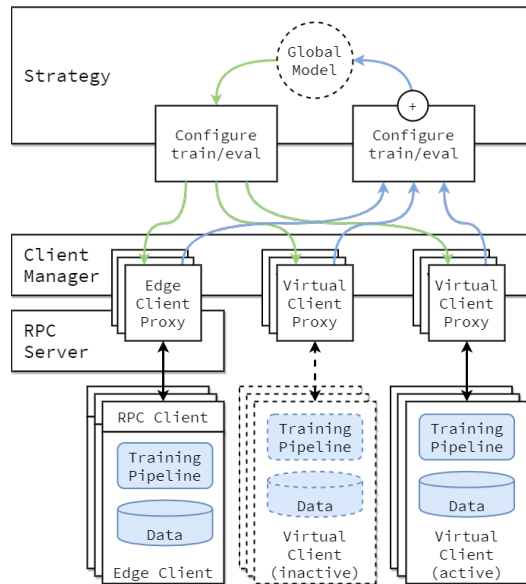


Figure 2. Flower core framework architecture with both Edge Client Engine and Virtual Client Engine. Edge clients live on real edge devices and communicate with the server over RPC. Virtual clients on the other hand consume close to zero resources when inactive and only load model and data into memory when the client is being selected for training or evaluation.

process over a set of available clients. Local computations are executed on individual clients and have access to actual data used for training or evaluation of model parameters.

The architecture of the Flower core framework reflects that perspective and enables researchers to experiment with building blocks, both on the global and on the local level. Global logic for client selection, configuration, parameter update aggregation, and federated or centralized model evaluation can be expressed through the *Strategy* abstraction. An implementation of the *Strategy* abstraction represents a single FL algorithm and Flower provides tested reference implementations of popular FL algorithms such as FedAvg (McMahan et al., 2017) or FedYogi (Reddi et al., 2021) (summarized in table 1). Local logic on the other hand is mainly concerned with model training and evaluation on local data partitions. Flower acknowledges the breadth and diversity of existing ML pipelines and offers ML framework-agnostic ways to federate these, either on the *Flower Protocol* level or using the high-level *Client* abstraction. Figure 2 illustrates those components.

The Flower core framework implements the necessary infrastructure to run these workloads at scale. On the server side, there are three major components involved: the *ClientManager*, the FL loop, and a (user customizable) *Strategy*. Server components sample clients from the *ClientManager*, which manages a set of *ClientProxy* objects, each representing a single client connected to the server. They are responsible for sending and receiving *Flower Protocol* mes-

sages to and from the actual client. The FL loop is at the heart of the FL process: it orchestrates the entire learning process. It does not, however, make decisions about *how* to proceed, as those decisions are delegated to the currently configured *Strategy* implementation.

In summary, the FL loop asks the *Strategy* to configure the next round of FL, sends those configurations to the affected clients, receives the resulting client updates (or failures) from the clients, and delegates result aggregation to the *Strategy*. It takes the same approach for both federated training and federated evaluation, with the added capability of server-side evaluation (again, via the *Strategy*). The client side is simpler in the sense that it only waits for messages from the server. It then reacts to the messages received by calling user-provided training and evaluation functions.

A distinctive property of this architecture is that the server is unaware of the nature of connected clients, which allows to train models across heterogeneous client platforms and implementations, including workloads comprised of clients connected through different communication stacks. The framework manages underlying complexities such as connection handling, client life cycle, timeouts, and error handling in an for the researcher.

3.4 Virtual Client Engine

Built into Flower is the Virtual Client Engine (VCE): a tool that enables the virtualization of Flower Clients to maximise utilization of the available hardware. Given a pool of clients, their respective compute and memory budgets (e.g. number of CPUs, VRAM requirements) and, the FL-specific hyperparameters (e.g. number of clients per round), the VCE launches Flower Clients in a resource-aware manner. The VCE will schedule, instantiate and run the Flower Clients in a transparent way to the user and the Flower Server. This property greatly simplifies parallelization of jobs, ensuring the available hardware is not underutilised and, enables porting the same FL experiment to a wide varying of setups without reconfiguration: a desktop machine, a single GPU rack or multi-node GPU cluster. The VCE therefore becomes a key module inside the Flower framework enabling running large scale FL workloads with minimal overhead in a scalable manner.

3.5 Edge Client Engine

Flower is designed to be open source, extendable and framework and device agnostic. Some devices suitable for lightweight FL workloads such as Raspberry Pi or NVIDIA Jetson require minimal or no special configuration. These Python-enabled embedded devices can readily be used as Flower Clients. On the other hand, commodity devices such as smartphones require a more strict, limited and sometimes proprietary software stack to run ML workloads. To circum-

Table 2. Comparison of different FL frameworks.

	TFF	Syft	FedScale	LEAF	Flower
Single-node simulation	✓	✓	✓	✓	✓
Multi-node execution	*	✓	(✓)***		✓
Scalability	*		**		✓
Heterogeneous clients		(✓)***	**		✓
ML framework-agnostic		****	****		✓
Communication-agnostic					✓
Language-agnostic					✓
Baselines			✓	✓	*

Labels: * Planned / ** Only simulated

*** Only Python-based / **** Only PyTorch and/or TF/Keras

vent this limitation, Flower provides a low-level integration by directly handling *Flower Protocol* messages on the client.

3.6 Secure Aggregation

In FL the server does not have direct access to a client’s data. To further protect clients’ local data, Flower provides implementation of both SecAgg (Bonawitz et al., 2017) and SecAgg+ (Bell et al., 2020) protocols for a semi-honest threat model. The Flower secure aggregation implementation satisfies five goals: usability, flexibility, compatibility, reliability and efficiency. The execution of secure aggregation protocols is independent of any special hardware and ML framework, robust against client dropouts, and has lower theoretical overhead for both communication and computation than other traditional multi-party computation secure aggregation protocol, which will be shown in 5.5.

3.7 FL Framework Comparison

We compare Flower to other FL toolkits, namely TFF (Google, 2020), Syft (Ryffel et al., 2018), FedScale (Lai et al., 2021) and LEAF (Caldas et al., 2018). Table 2 provides an overview, with a more detailed description of those properties following thereafter.

Single-node simulation enables simulation of FL systems on a single machine to investigate workload performance without the need for a multi-machine system. Supported by all frameworks.

Multi-node execution requires network communication between server and clients on different machines. Multi-machine execution is currently supported by Syft and Flower. FedScale supports multi-machine simulation (but not real deployment), TFF plans multi-machine deployments.

Scalability is important to derive experimental results that generalize to large cohorts. Single-machine simulation is limited because workloads including a large number of clients often exhibit vastly different properties. TFF and LEAF are, at the time of writing, constrained to single-machine simulations. FedScale can simulate clients on mul-

tiples machines, but only scales to 100 concurrent clients. Syft is able to communicate over the network, but only by connecting to data holding clients that act as servers themselves, which limits scalability. In Flower, data-holding clients connect to the server which allows workloads to scale to millions of clients, including scenarios that require full control over when connections are being opened and closed. Flower also includes a virtual client engine for large-scale multi-node simulations.

Heterogeneous clients refers to the ability to run workloads comprised of clients running on different platforms using different languages, all in the same workload. FL targeting edge devices will clearly have to assume pools of clients of many different types (e.g., phone, tablet, embedded). Flower supports such heterogeneous client pools through its language-agnostic and *communication-agnostic* client-side integration points. It is the only framework in our comparison that does so, with TFF and Syft expecting a framework-provided client runtime, whereas FedScale and LEAF focus on Python-based simulations.

ML framework-agnostic toolkits allow researchers and users to leverage their previous investments in existing ML frameworks by providing universal integration points. This is a unique property of Flower: the ML framework landscape is evolving quickly (e.g., JAX (Bradbury et al., 2018), PyTorch Lightning (W. Falcon, 2019)) and therefore the user should choose which framework to use for their local training pipelines. TFF is tightly coupled with TensorFlow and experimentally supports JAX, LEAF also has a dependency on TensorFlow, and Syft provides hooks for PyTorch and Keras, but does not integrate with arbitrary tools.

Language-agnostic describes the capability to implement clients in a variety of languages, a property especially important for research on mobile and emerging embedded platforms. These platforms often do not support Python, but rely on specific languages (Java on Android, Swift on iOS) for idiomatic development, or native C++ for resource constrained embedded devices. Flower achieves a fully language-agnostic interface by offering protocol-level integration. Other frameworks are based on Python, with some of them indicating a plan to support Android and iOS (but not embedded platforms) in the future.

Baselines allow the comparison of existing methods with new FL algorithms. Having existing implementations at ones disposal can greatly accelerate research progress. LEAF and FedScale come with a number of benchmarks built-in with different datasets. TFF provides libraries for constructing baselines with some datasets. Flower currently implements a number of FL methods in the context of popular ML benchmarks, e.g., a federated training of CIFAR-10 (Krizhevsky et al., 2005) image classification, and has initial port of LEAF datasets such as FEMNIST and Shake-

spare (Caldas et al., 2018).

4 IMPLEMENTATION

Flower has an extensive implementation of FL averaging algorithms, a robust communication stack, and various examples of deploying Flower on real and simulated clients. Due to space constraints, we only focus on some of the implementation details in this section and refer the reader to the Flower GitHub repository for more details.

Communication stack. FL requires stable and efficient communication between clients and server. The Flower communication protocol is currently implemented on top of bi-directional gRPC (Foundation) streams. gRPC defines the types of messages exchanged and uses compilers to then generate efficient implementations for different languages such as Python, Java, or C++. A major reason for choosing gRPC was its efficient binary serialization format, which is especially important on low-bandwidth mobile connections. Bi-directional streaming allows for the exchange of multiple message without the overhead incurred by re-establishing a connection for every request/response pair.

Serialization. Independent of communication stack, Flower clients receive instructions (messages) as raw byte arrays (either via the network or through other means, for example, inter-process communication), deserialize the instruction, and execute the instruction (e.g., training on local data). The results are then serialized and communicated back to the server. Note that a client communicates with the server through language-independent messages and can thus be implemented in a variety of programming languages, a key property to enable real on-device execution. The user-accessible byte array abstraction makes Flower uniquely serialization-agnostic and enables users to experiment with custom serialization methods, for example, gradient compression or encryption.

Alternative communication stacks. Even though the current implementation uses gRPC, there is no inherent reliance on it. The internal Flower server architecture uses modular abstractions such that components that are not tied to gRPC are unaware of it. This enables the server to support user-provided RPC frameworks and orchestrate workloads across heterogeneous clients, with some connected through gRPC, and others through other RPC frameworks.

ClientProxy. The abstraction that enables communication-agnostic execution is called *ClientProxy*. Each *ClientProxy* object registered with the *ClientManager* represents a single client that is available to the server for training or evaluation. Clients which are offline do not have an associated *ClientProxy* object. All server-side logic (client configuration, receiving results from clients) is built against the *ClientProxy* abstraction.

One key design decision that makes Flower so flexible is that *ClientProxy* is an abstract interface, not an implementation. There are different implementations of the *ClientProxy* interface, for example, *GrpcClientProxy*. Each implementation encapsulates details on how to communicate with the actual client, for example, to send messages to an actual edge device using gRPC.

Virtual Client Engine (VCE). Resource consumption (CPU, GPU, RAM, VRAM, etc.) is the major bottleneck for large-scale experiments. Even a modestly sized model easily exhausts most systems if kept in memory a million times. The VCE enables large-scale single-machine or multi-machine experiments by executing workloads in a resource-aware fashion that either increases parallelism for better wall-clock time or to enable large-scale experiments on limited hardware resources. It creates a *ClientProxy* for each client, but defers instantiation of the actual client object (including local model and data) until the resources to execute the client-side task (training, evaluation) become available. This avoids having to keep multiple client-side models and datasets in memory at any given point in time.

VCE builds on the Ray (Moritz et al., 2018) framework to schedule the execution of client-side tasks. In case of limited resources, Ray can sequence the execution of client-side computations, thus enabling a much larger scale of experiments on common hardware. The capability to perform FL at scale will unlock new research opportunities as results obtained in small-scale experiments often do not generalize well to large-cohort settings.

5 FRAMEWORK EVALUATION

In this section we evaluate Flower’s capabilities in supporting both research and implementations of real-world FL workloads. Our evaluation focuses on three main aspects:

- **Scalability:** We show that Flower can (a) efficiently make use of available resources in single-machine simulations and (b) run experiments with millions of clients whilst sampling thousands in each training.
- **Heterogeneity:** We show that Flower can be deployed in real, heterogeneous devices commonly found in cross-device scenario and how it can be used to measure system statistics.
- **Realism:** We show through a case study how Flower can throw light on the performance of FL under heterogeneous clients with different computational and network capabilities.
- **Privacy:** Finally, we show how our implementation of Secure Aggregation matches the expected theoretical overhead as expected.

5.1 Large-Scale Experiment

Federated Learning receives most of its power from its ability to leverage data from millions of users. However, selecting large numbers of clients in each training round does not necessarily translate into faster convergence times. In fact, as observed in (McMahan et al., 2017), there is usually an empirical threshold for which if we increase the number of participating clients per round beyond that point, convergence will be slower. By allowing experiments to run at mega-scales, with thousands of active clients per round, Flower gives us the opportunity to empirically find such threshold for any task at hand.

To show this ability, in this series of experiments we use Flower to fine-tune a network on data from 15M users using different numbers of clients per round. More specifically, we fine-tune a Transformer network to correctly predict Amazon book ratings based on text reviews from users.

Experimental Setup. We choose to use Amazon’s Book Reviews Dataset (Ni et al., 2019) which contains over 51M reviews from 15M different users. Each review from a given user contains a textual review of a book along with its given rank (1-5). We fine-tune the classifier of a pre-trained DistilBERT model (Sanh et al., 2019) to correctly predict ranks based on textual reviews. For each experiment we fix the number of clients being sampled in each round (from 10 to 1000) and aggregate models using FedAvg. We test the aggregated model after each round on a fixed set of 1M clients. Convergence curves are reported in Figure 3 all our experiments were run using two NVIDIA V100 GPUs on a 22-cores of an Intel Xeon Gold 6152 (2.10GHz) CPU.

Results. Figure 3 shows the expected initial speed-up in convergence when selecting 10 to 500 clients per round in each experiment. However, if we decide to sample 1k clients in each round, we notice an increase in convergence time. Intuitively, this behaviour is caused by clients’ data having very different distributions; making it difficult for simple Aggregation Strategies such as FedAvg to find a suitable set of weights.

5.2 Single Machine Experiments

One of our strongest claims in this paper is that Flower can be effectively used in Research. For this to be true, Flower needs to be fast at providing reliable results when experimenting new ideas, e.g. a new aggregation strategy.

In this experiment, we provide a head-to-head comparison in term of training times between Flower and the four main FL frameworks, namely FedScale, TFF, FedJax and the original LEAF, when training with different FL setups.

Experimental Setup. We consider all three FL setups proposed by (Caldas et al., 2018) when training a CNN model

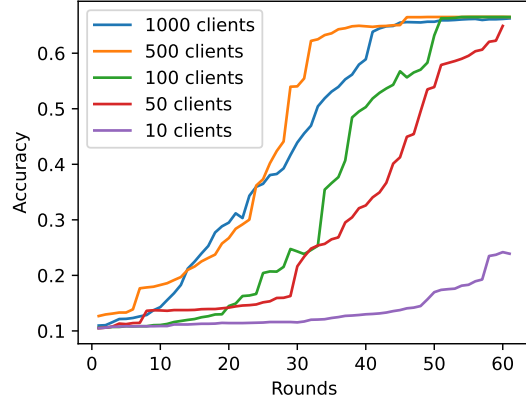


Figure 3. Flower scales to even 15M user experiments. Each curve shows successful convergence of the DistilBERT model under varying amounts of clients per round, with the exception of the two smallest client sizes: 50 and 10.

to correctly classify characters from the *FEMNIST* dataset. More specifically, we consider the scenarios where the number of clients (c) and local epochs per round change (l) vary. The total number of rounds and total number of clients are kept constant at 2000 and 179, respectively. To allow for a fair comparison, We run all our experiments using eight cores of an Intel Xeon E5-2680 CPU (2.40GHz) equipped with two NVIDIA RTX2080 GPUs and 20GB of RAM.

Results. Figure 4 shows the impact of choosing different FL frameworks for the various tasks. On our first task, when training using three clients per round ($c = 3$) for one local epoch ($l = 1$), FedJax finishes training first (05:18), LEAF finishes second (44:39) followed by TFF (58:29) and Flower (59:19). In this simple case, the overhead of having a multi-task system, like the Virtual Client Engine (VCE), causes Flower to slightly under-perform in comparison to loop-based simulators, like LEAF.

However, the benefits of having a VCE become more evident if we train on more realistic scenarios. When increasing the number of clients per round to 35 while keeping the single local epoch, we notice that Flower (230:18) is still among the fastest frameworks. Since the number of local epochs is still one, most of the overhead comes from loading data and models into memory rather than performing real training, hence the similarity those LEAF and Flower.

The VCE allows us to specify the amount of GPU memory we want to associate with each client, this allows for more efficient data and model loading of different clients on the same GPU, making the overall training considerably faster. In fact, when we substantially increase the amount of work performed by each client to 100 local epochs, while fixing the number of active client to 3, we see a significant saving in training time. In this task Flower outperforms all other. It

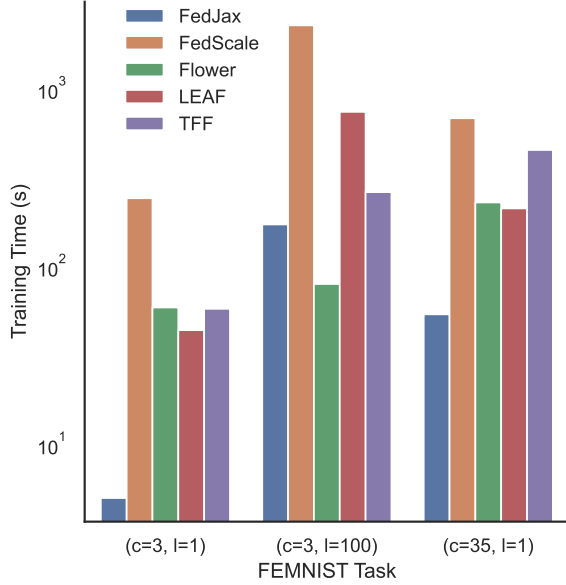


Figure 4. Training times (log scale in second) comparison of different FEMNIST tasks between different FL frameworks.

completes the task in just about 80 minutes, while the second best performing framework (FedJax) takes over twice as long (over 173 minutes).

It is also important to acknowledge the two extreme training times we see in this experiment. FedJax seems to be very efficient when training on few (1) local epochs; however, in scenarios where communication-efficiency is key and larger number of local epochs are required, FedJax performance slightly degrades. FedScale, on the other hands, consistently showed high training times across all training scenarios. We believe this apparent inefficiency to be associated with network overheads that are usually unnecessary in a single-computer simulation.

5.3 Flower enables FL evaluation on real devices

Flower can assist researchers in quantifying the system costs associated with running FL on real devices and to identify bottlenecks in real-world federated training. In this section, we present the results of deploying Flower on six types of heterogeneous real-world mobile and embedded devices, including Java-based Android smartphones and Python-based Nvidia Jetson series devices and Raspberry Pi.

Experiment Setup. We run the Flower server configured with the FedAvg strategy and host it on a cloud virtual machine. Python-based Flower clients are implemented for Nvidia Jetson series devices (Jetson Nano, TX2, NX, AGX) and Raspberry Pi, and trained using TensorFlow as the ML framework on each client. On the other hand, Android smartphones currently do not have extensive on-device training support with TensorFlow or PyTorch. To counter this issue, we leverage TensorFlow Lite to implement Flower clients

on Android smartphones in Java. While TFLite is primarily designed for on-device inference, we leverage its capabilities to do on-device model personalization to implement a FL client application (Lite, 2020). The source code for both implementations is available in the Flower repository.

Results. Figure 5 shows the system metrics associated with training a DeepConvLSTM (Singh et al., 2021) model for a human activity recognition task on Python-enabled Jetson and Raspberry Pi devices. We used the RealWorld dataset (Sztyler & Stuckenschmidt, 2016) consisting of time-series data from accelerometer and gyroscope sensors on mobile devices, and partitioned it across 10 mobile clients. The first takeaway from our experiments is that we could deploy Flower clients on these heterogeneous devices, without requiring any modifications in the client-side Flower code. The only consideration was to ensure that a compatible ML framework (e.g., TensorFlow) is installed on each client. Secondly, we show in Figure 5 how FL researchers can deploy and quantify the training time and energy consumption of FL on various heterogeneous devices and processors. Here, the FL training time is aggregated over 40 rounds, and includes the time taken to perform local 10 local epochs of SGD on the client, communicating model parameters between the server and the client, and updating the global model on the server. By comparing the relative energy consumption and training times across various devices, FL researchers can devise more informed client selection policies that can tradeoff between FL convergence time and overall energy consumption. For instance, choosing Jetson Nano-CPU based FL clients over Raspberry Pi clients may increase FL convergence time by 10 minutes, however it reduces the overall energy consumption by almost 60%.

Next, we illustrate how Flower can enable fine-grained profiling of FL on real devices. We deploy Flower on 10 Android clients to train a model with 2 convolutional layers and 3 fully-connected layers (Flower, 2021) on the CIFAR-10 dataset. TensorFlow Lite is used as the training ML framework on the devices. We measure the time taken for various FL operations, such as local SGD training, communication between the server and client, local evaluation on the client, and the overhead due to the Flower framework.

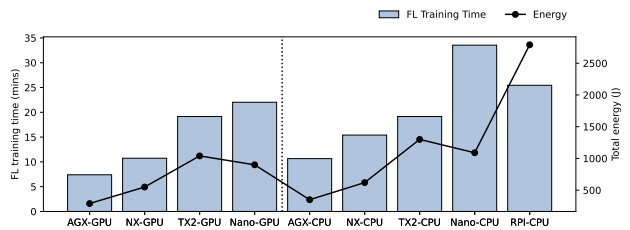


Figure 5. Flower enables quantifying the system performance of FL on mobile and embedded devices. Here we report the training times and energy consumption associated with running FL on CPUs and GPUs of various embedded devices.

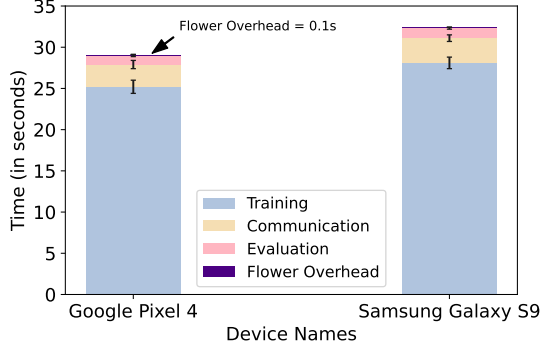


Figure 6. Flower enables fine-grained profiling of FL performance on real devices. The framework overhead is $<100\text{ms}$ per round.

Table 3. Effect of computational heterogeneity on FL training times. Using Flower, we can compute a hardware-specific cutoff τ (in minutes) for each processor, and find a balance between FL accuracy and training time. $\tau = 0$ indicates no cutoff time.

	GPU	CPU ($\tau = 0$)	CPU ($\tau = 2.23$)	CPU ($\tau = 1.99$)
Accuracy	0.67	0.67	0.66	0.63
Training time (mins)	80.32	102 ($1.27\times$)	89.15 ($1.11\times$)	80.34 ($1.0\times$)

The overhead includes converting model gradients to GRPC-compatible buffers and vice-versa, to enable communication between Java FL clients and a Python FL server. In Figure 6, we report the mean latency of various FL operations over 40 rounds on two types of Android devices: Google Pixel 4 and Samsung Galaxy S9. We observe that on both devices, local training remains the most time-consuming operation, and that the total system overhead of the Flower framework is less than 100ms per round.

5.4 Realism in Federated Learning

Flower facilitates the deployment of FL on real-world devices. While this property is beneficial for production-grade systems, can it also assist researchers in developing better federated optimization algorithms? In this section, we study two realistic scenarios of FL deployment.

Computational Heterogeneity across Clients. In real-world, FL clients will have vastly different computational capabilities. While newer smartphones are now equipped with mobile GPUs, other phones or wearable devices may have a much less powerful processor. How does this computational heterogeneity impact FL?

For this experiment, we use a Nvidia Jetson TX2 as the client device, which has a Pascal GPU and six CPU cores. We train a ResNet18 model on the CIFAR-10 dataset in a federated setting with 10 total Jetson TX2 clients and 40 rounds of training. In Table 3, we observe that if Jetson TX2

CPU clients are used for federated training (local epochs $E=10$), the FL process would take $1.27\times$ more time to converge as compared to training on Jetson TX2 GPU clients.

Once we obtain this quantification of computational heterogeneity using Flower, we can design better federated optimization algorithms. As an example, we implemented a modified version of FedAvg where each client device is assigned a cutoff time (τ) after which it must send its model parameters to the server, irrespective of whether it has finished its local epochs or not. This strategy has parallels with the FedProx algorithm (Li et al., 2018) which also accepts partial results from clients. However, the key advantage of Flower’s on-device training capabilities is that we can accurately measure and assign a *realistic* processor-specific cutoff time for each client. For example, we measure that on average it takes 1.99 minutes to complete a FL round on the TX2 GPU. We then set the same time as a cutoff for CPU clients ($\tau = 1.99$ mins) as shown in Table 3. This ensures that we can obtain faster convergence even in the presence of CPU clients, at the expense of a 4% accuracy drop. With $\tau = 2.23$, a better balance between accuracy and convergence time could be obtained for CPU clients.

Heterogeneity in Network Speeds. An important consideration for any FL system is to choose a set of participating clients in each training round. In the real-world, clients are distributed across the world and vary in their download and upload speeds. Hence, it is critical for any FL system to study how client selection can impact the overall FL training time. We now present an experiment with 40 clients collaborating to train a 4-layer deep CNN model for the FashionMNIST dataset. More details about the dataset and network architecture are presented in the Appendix.

Using Flower, we instantiate 40 clients on a cloud platform and fix the download and upload speeds for each client using the WONDERSHAPER library. Each client is representative of a country and its download and upload speed is set based on a recent market survey of 4G and 5G speeds in different countries (OpenSignal, 2020).

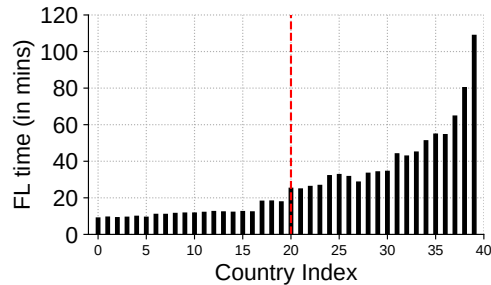


Figure 7. Effect of network heterogeneity in clients on FL training time. Using this quantification, we designed a new client sampling strategy called FedFS (detailed in the Appendix).

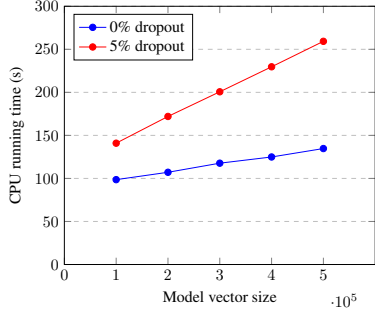


Figure 8. Performance of Secure Aggregation. Running time of server with increasing vector size

The x-axis of Figure 7 shows countries arranged in descending order of their network speeds: country indices 1-20 represent the top 20 countries based on their network speeds (mean download speed = 40.1Mbps), and indices 21-40 are the bottom 20 countries (mean download speed = 6.76Mbps). We observe that if all clients have the network speeds corresponding to Country 1 (Canada), the FL training finishes in 8.9 mins. As we include slower clients in FL, the training time gradually increases, with a major jump around index = 17. On the other extreme, for client speeds corresponding to Country 40 (Iraq), the FL training takes 108 minutes.

There are two key takeaways from this experiment: a) Using Flower, we can profile the training time of any FL algorithm under scenarios of network heterogeneity, b) we can leverage these insights to design sophisticated client sampling techniques. For example, during subsequent rounds of federated learning, we could monitor the number of samples each client was able to process during a given time window and increase the selection probability of slow clients to balance the contributions of fast and slow clients to the global model. The FedFS strategy detailed in the appendix works on this general idea, and reduces the convergence time of FL by up to 30% over the FedAvg random sampling approach.

5.5 Secure Aggregation Overheads

Privacy is one of the cornerstones in FL, which inevitably generates computational overhead during training. In hardware-constrained systems, such as cross-device FL, it is desirable not only to be able to measure such overheads, but also to make sure that security protocols are well implemented and follow the expected protocol described in the original papers. Flower’s implementation of Secure Aggregation, named Salvia, is based on the SecAgg (Bonawitz et al., 2017) and SecAgg+ (Bell et al., 2020) protocols as described in Section 3.6. To verify that Salvia’s behavior matches the expected theoretical complexity, we evaluate its impact on server-side computation and communication overhead with the model vector size and clients dropouts.

Experiment Setup. The FL simulations run on a Linux

system with an Intel Xeon E-2136 CPU (3.30GHz), with 256 GB of RAM. In our simulations, all entries of our local vectors are of size 24 bits. We ignore communication latency. Moreover, all dropouts simulated happen after stage 2, i.e. Share Keys Stage. This is because this imposes the most significant overhead as the server not only needs to regenerate dropped-out clients’ secrets, but also compute their pairwise masks generated between their neighbours.

For our simulations, the n and t parameters of the t -out-of- n secret-sharing scheme are set to 51 and 26, respectively. These parameters are chosen to reference SecAgg+’s proven correctness and security guarantees, where we can tolerate up to 5% dropouts and 5% corrupted clients with correctness holding with probability $1 - 2^{-20}$ and security holding with probability $1 - 2^{-40}$.

Results. Fixing the number of sampled clients to 100, we plotted CPU running times through aggregating a vector of size 100k entries to aggregating one of size 500k entries in Figure 8. We also measured how the performance would change after client dropouts by repeating the same experiments with a 5% client dropout.

Both the running times and total data transfer of the server increase linearly with the model vector size as the operations involving model vectors are linear to the vectors’ sizes, e.g. generating masks, sending vectors. We also note the server’s running time increases when there are 5% clients dropping out, as the server has to perform extra computation to calculate all k pairwise masks for each client dropped. Lastly, we observe that the total data transferred of the server remains unchanged with client dropouts as each client only communicates with the server plus exactly k neighbors, regardless of the total number of clients and dropouts. We conclude that all our experimental data matches the expected complexities of SecAgg and SecAgg+.

6 CONCLUSION

We have presented Flower – a novel framework that is specifically designed to advance FL research by enabling heterogeneous FL workloads at scale. Although Flower is broadly useful across a range of FL settings, we believe that it will be a true *game-changer* for reducing the disparity between FL research and real-world FL systems. Through the provided abstractions and components, researchers can federated existing ML workloads (regardless of the ML framework used) and transition these workloads from large-scale simulation to execution on heterogeneous edge devices. We further evaluate the capabilities of Flower in experiments that target both scale and systems heterogeneity by scaling FL up to 15M clients, providing head-to-head comparison between different FL frameworks for single-computer experiments, measuring FL energy consumption on a cluster of Nvidia

Jetson TX2 devices, optimizing convergence time under limited bandwidth, and illustrating a deployment of Flower on a range of Android mobile devices in the AWS Device Farm. Flower is open-sourced under Apache 2.0 License and we look forward to more community contributions to it.

REFERENCES

- Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M., Kudlur, M., Levenberg, J., Monga, R., Moore, S., Murray, D. G., Steiner, B., Tucker, P., Vasudevan, V., Warden, P., Wicke, M., Yu, Y., and Zheng, X. Tensorflow: A system for large-scale machine learning. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, pp. 265–283, 2016a. URL <https://www.usenix.org/system/files/conference/osdi16/osdi16-abadi.pdf>.
- Abadi, M., Chu, A., Goodfellow, I., McMahan, B., Mironov, I., Talwar, K., and Zhang, L. Deep learning with differential privacy. In *23rd ACM Conference on Computer and Communications Security (ACM CCS)*, pp. 308–318, 2016b. URL <https://arxiv.org/abs/1607.00133>.
- Bell, J. H., Bonawitz, K. A., Gascón, A., Lepoint, T., and Raykova, M. Secure single-server aggregation with (poly) logarithmic overhead. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, pp. 1253–1269, 2020.
- Bhagoji, A. N., Chakraborty, S., Mittal, P., and Calo, S. B. Analyzing federated learning through an adversarial lens. *CoRR*, abs/1811.12470, 2018. URL <http://arxiv.org/abs/1811.12470>.
- Bonawitz, K., Ivanov, V., Kreuter, B., Marcedone, A., McMahan, H. B., Patel, S., Ramage, D., Segal, A., and Seth, K. Practical secure aggregation for privacy-preserving machine learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS ’17*, pp. 1175–1191, New York, NY, USA, 2017. ACM. ISBN 978-1-4503-4946-8. doi: 10.1145/3133956.3133982. URL <http://doi.acm.org/10.1145/3133956.3133982>.
- Bonawitz, K., Eichner, H., Grieskamp, W., Huba, D., Ingerman, A., Ivanov, V., Kiddon, C. M., Konečný, J., Mazzocchi, S., McMahan, B., Overveldt, T. V., Petrou, D., Ramage, D., and Roselander, J. Towards federated learning at scale: System design. In *SysML 2019*, 2019. URL <https://arxiv.org/abs/1902.01046>. To appear.
- Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., and Zhang, Q. JAX: composable transformations of Python+NumPy programs, 2018. URL <http://github.com/google/jax>.
- Caldas, S., Duddu, S. M. K., Wu, P., Li, T., Konečný, J., McMahan, H. B., Smith, V., and Talwalkar, A. Leaf: A benchmark for federated settings. *arXiv preprint arXiv:1812.01097*, 2018.
- Chahal, K. S., Grover, M. S., and Dey, K. A hitchhiker’s guide on distributed training of deep neural networks. *CoRR*, abs/1810.11787, 2018. URL <http://arxiv.org/abs/1810.11787>.
- Chowdhery, A., Warden, P., Shlens, J., Howard, A., and Rhodes, R. Visual wake words dataset. *CoRR*, abs/1906.05721, 2019. URL <http://arxiv.org/abs/1906.05721>.
- Dean, J., Corrado, G. S., Monga, R., Chen, K., Devin, M., Le, Q. V., Mao, M. Z., Ranzato, M., Senior, A., Tucker, P., Yang, K., and Ng, A. Y. Large scale distributed deep networks. In *Proceedings of the 25th International Conference on Neural Information Processing Systems - Volume 1, NIPS’12*, pp. 1223–1231, USA, 2012. Curran Associates Inc. URL <http://dl.acm.org/citation.cfm?id=2999134.2999271>.
- Dryden, N., Jacobs, S. A., Moon, T., and Van Essen, B. Communication quantization for data-parallel training of deep neural networks. In *Proceedings of the Workshop on Machine Learning in High Performance Computing Environments, MLHPC ’16*, pp. 1–8, Piscataway, NJ, USA, 2016. IEEE Press. ISBN 978-1-5090-3882-4. doi: 10.1109/MLHPC.2016.4. URL <https://doi.org/10.1109/MLHPC.2016.4>.
- Flower. Model architecture for android devices. https://github.com/adap/flower/blob/main/examples/android/tflite_convertor/convert_to_tflite.py, 2021.
- Foundation, C. N. C. grpc: A high performance, open-source universal rpc framework. URL <https://grpc.io>. Accessed: 2020-03-25.
- Fromm, J., Patel, S., and Philipose, M. Heterogeneous bitwidth binarization in convolutional neural networks. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems, NIPS’18*, pp. 4010–4019, Red Hook, NY, USA, 2018. Curran Associates Inc.
- Georgiev, P., Lane, N. D., Mascolo, C., and Chu, D. Accelerating mobile audio sensing algorithms through on-chip GPU offloading. In Choudhury, T., Ko, S. Y., Campbell, A., and Ganesan, D. (eds.), *Proceedings of the*

- 15th Annual International Conference on Mobile Systems, Applications, and Services, MobiSys'17, Niagara Falls, NY, USA, June 19-23, 2017, pp. 306–318. ACM, 2017. doi: 10.1145/3081333.3081358. URL <https://doi.org/10.1145/3081333.3081358>.
- Google. Tensorflow federated: Machine learning on decentralized data. <https://www.tensorflow.org/federated>, 2020. accessed 25-Mar-20.
- Hard, A., Rao, K., Mathews, R., Ramaswamy, S., Beaufays, F., Augenstein, S., Eichner, H., Kiddon, C., and Ramage, D. Federated learning for mobile keyboard prediction, 2019.
- Jia, X., Song, S., He, W., Wang, Y., Rong, H., Zhou, F., Xie, L., Guo, Z., Yang, Y., Yu, L., Chen, T., Hu, G., Shi, S., and Chu, X. Highly scalable deep learning training system with mixed-precision: Training imagenet in four minutes. *CoRR*, abs/1807.11205, 2018. URL <http://arxiv.org/abs/1807.11205>.
- Krizhevsky, A., Nair, V., and Hinton, G. Cifar-10 (canadian institute for advanced research). *Online*, 2005. URL <http://www.cs.toronto.edu/~kriz/cifar.html>.
- Lai, F., Dai, Y., Zhu, X., and Chowdhury, M. FedScale: Benchmarking model and system performance of federated learning. *arXiv preprint arXiv:2105.11367*, 2021.
- Lee, T., Lin, Z., Pushp, S., Li, C., Liu, Y., Lee, Y., Xu, F., Xu, C., Zhang, L., and Song, J. Occlumency: Privacy-preserving remote deep-learning inference using sgx. In *The 25th Annual International Conference on Mobile Computing and Networking, MobiCom '19*, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450361699. doi: 10.1145/3300061.3345447. URL <https://doi.org/10.1145/3300061.3345447>.
- Li, T., Sahu, A. K., Zaheer, M., Sanjabi, M., Talwalkar, A., and Smith, V. Federated optimization in heterogeneous networks. *arXiv preprint arXiv:1812.06127*, 2018.
- Li, T., Sanjabi, M., and Smith, V. Fair resource allocation in federated learning. *arXiv preprint arXiv:1905.10497*, 2019.
- Li, T., Sahu, A. K., Zaheer, M., Sanjabi, M., Talwalkar, A., and Smith, V. Federated optimization in heterogeneous networks, 2020.
- LiKamWa, R., Hou, Y., Gao, J., Polansky, M., and Zhong, L. Redeye: Analog convnet image sensor architecture for continuous mobile vision. In *Proceedings of the 43rd International Symposium on Computer Architecture*, ISCA '16, pp. 255–266. IEEE Press, 2016. ISBN 9781467389471. doi: 10.1109/ISCA.2016.31. URL <https://doi.org/10.1109/ISCA.2016.31>.
- Lite, T. On-device model personalization. <https://blog.tensorflow.org/2019/12/example-on-device-model-personalization.html>, 2020.
- Malekzadeh, M., Athanasakis, D., Haddadi, H., and Livshits, B. Privacy-preserving bandits, 2019.
- McMahan, B., Moore, E., Ramage, D., Hampson, S., and y Arcas, B. A. Communication-efficient learning of deep networks from decentralized data. In Singh, A. and Zhu, X. J. (eds.), *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics, AISTATS 2017, 20-22 April 2017, Fort Lauderdale, FL, USA*, volume 54 of *Proceedings of Machine Learning Research*, pp. 1273–1282. PMLR, 2017. URL <http://proceedings.mlr.press/v54/mcmahan17a.html>.
- Moritz, P., Nishihara, R., Wang, S., Tumanov, A., Liaw, R., Liang, E., Elibol, M., Yang, Z., Paul, W., Jordan, M. I., and Stoica, I. Ray: A distributed framework for emerging ai applications, 2018.
- Ni, J., Li, J., and McAuley, J. Justifying recommendations using distantly-labeled reviews and fine-grained aspects. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 188–197, 2019.
- OpenSignal. The state of mobile network experience 2020: One year into the 5g era. <https://www.opensignal.com/reports/2020/05/global-state-of-the-mobile-network>, 2020. accessed 10-Oct-20.
- Reddi, S., Charles, Z., Zaheer, M., Garrett, Z., Rush, K., Konečný, J., Kumar, S., and McMahan, H. B. Adaptive federated optimization, 2021.
- Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., et al. Imagenet large scale visual recognition challenge. *International journal of computer vision*, 115(3): 211–252, 2015.
- Ryffel, T., Trask, A., Dahl, M., Wagner, B., Mancuso, J., Rueckert, D., and Passerat-Palmbach, J. A generic framework for privacy preserving deep learning. *CoRR*, abs/1811.04017, 2018. URL <http://arxiv.org/abs/1811.04017>.
- Sanh, V., Debut, L., Chaumond, J., and Wolf, T. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.

Sergeev, A. and Balso, M. D. Horovod: fast and easy distributed deep learning in tensorflow. *CoRR*, abs/1802.05799, 2018. URL <http://arxiv.org/abs/1802.05799>.

Singh, S. P., Sharma, M. K., Lay-Ekuakille, A., Gangwar, D., and Gupta, S. Deep convlstm with self-attention for human activity decoding using wearable sensors. *IEEE Sensors Journal*, 21(6):8575–8582, Mar 2021. ISSN 2379-9153. doi: 10.1109/jsen.2020.3045135. URL <http://dx.doi.org/10.1109/JSEN.2020.3045135>.

Smith, V., Chiang, C.-K., Sanjabi, M., and Talwalkar, A. S. Federated multi-task learning. In *Advances in Neural Information Processing Systems*, pp. 4424–4434, 2017.

Sztyler, T. and Stuckenschmidt, H. On-body localization of wearable devices: An investigation of position-aware activity recognition. In *2016 IEEE International Conference on Pervasive Computing and Communications (PerCom)*, pp. 1–9. IEEE Computer Society, 2016. doi: 10.1109/PERCOM.2016.7456521.

W. Falcon, e. a. Pytorch lightning, 2019. URL <https://github.com/williamFalcon/pytorch-lightning>.

Xiao, H., Rasul, K., and Vollgraf, R. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017.

Xie, C., Koyejo, S., and Gupta, I. Zeno: Distributed stochastic gradient descent with suspicion-based fault-tolerance. In Chaudhuri, K. and Salakhutdinov, R. (eds.), *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pp. 6893–6901, Long Beach, California, USA, 09–15 Jun 2019. PMLR. URL <http://proceedings.mlr.press/v97/xie19b.html>.

Yao, Y., Li, H., Zheng, H., and Zhao, B. Y. Latent backdoor attacks on deep neural networks. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS ’19*, pp. 2041–2055, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450367479. doi: 10.1145/3319535.3354209. URL <https://doi.org/10.1145/3319535.3354209>.

A APPENDIX

A.1 Survey on papers

From a systems perspective, a major bottleneck to FL research is the paucity of frameworks that support scalable

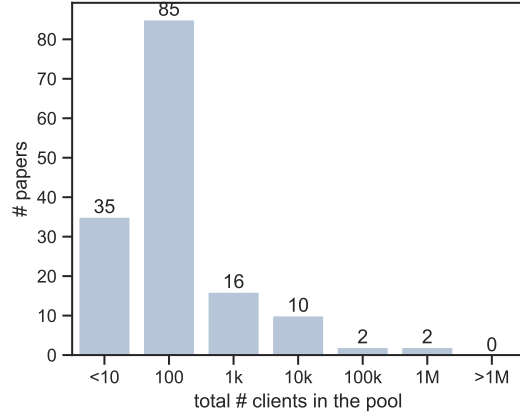


Figure 9. Histograms of the number of total FL clients used in FL research papers in the last two years. A vast majority of papers only use up to 100 clients.

execution of FL methods on mobile and edge devices. Fig. 9 shows the histograms of total number of clients in the FL pools in research papers. The research papers is gathered from Google Scholar that is related to federated learning from last 2 years which consists of total 150 papers in the survey. We excluded papers that are using the framework not available to reproduced the results. As we can see from the histogram, the majority of experiments only use up to 100 total clients, which usually on datasets such as CIFAR10 and ImageNet. There are only 3 papers using the dataset with a total clients pool up to 1 millions, and they are using the Reddit and Sentiment140 dataset from leaf (Caldas et al., 2018).

A.2 FedFS Algorithm

We introduce *Federating: Fast and Slow (FedFS)* to overcome the challenges arising from heterogeneous devices and non-IID data. *FedFS* acknowledges the difference in compute capabilities inherent in networks of mobile devices by combining partial work, importance sampling, and dynamic timeouts to enable clients to contribute equally to the global model.

Partial work. Given a (local) data set of size m_k on client k , a batch size of B , and the number of local training epochs E , FedAvg performs $E \frac{m_k}{B}$ (local) gradient updates $\theta^k \leftarrow \theta^k - \eta \nabla \ell(b; \theta^k)$ before returning θ^k to the server. The asynchronous setting treats the success of local update computation as binary. If a client succeeds in computing $E \frac{m_k}{B}$ mini-batch updates before reaching a timeout Δ , their weight update is considered by the server, otherwise it is discarded. The server then averages all successful $\theta_{k \in \{0, \dots, K\}}$ updates, weighted by m_k , the number of training examples on client k .

This is wasteful because a clients' computation might be discarded upon reaching Δ even if it was close to computing the full $E \frac{m_k}{B}$ gradient updates. We therefore apply the concept of partial work (Li et al., 2018) in which a client submits their locally updated θ_k upon reaching Δ along with c_k , the number of examples actually involved in computing θ_k , even if $c_k < E \frac{m_k}{B}$. The server averages by c_k , not m_k , because c_k can vary over different rounds and devices depending on a number of factors (device speed, concurrent processes, Δ , m_k , etc.).

Intuitively, this leads to more graceful performance degradation with smaller values for Δ . Even if Δ is set to an adversarial value just below the completion time of the fastest client, which would cause *FedAvg* to not consider any update and hence prevent convergence, *FedFS* would still progress by combining K partial updates. More importantly it allows devices which regularly discard their updates because of lacking compute capabilities to have their updates represented in the global model, which would otherwise overfit the data distribution on the subset of faster devices in the population.

Importance sampling. Partial work enables *FedFS* to leverage the observed values for c_k^r (with $r \in \{1, \dots, t\}$, the amount of work done by client k during all previous rounds up to the current round t) and $E^r m_k$ (with $r \in \{1, \dots, t\}$, the amount of work client k was maximally allowed to do during those rounds) for client selection during round $t + 1$. c and m can be measured in different ways depending on the use case. In vision, c_k^t could capture the number of image examples processed, whereas in speech c_k^t could measure the accumulated duration of all audio samples used for training on client k during round t . $c_k^t < E^t m_k$ suggests that client k was not able to compute $E^t \frac{m_k}{B}$ gradient updates within Δ_t , so its weight update θ_k^t has less of an impact on the global model θ compared to an update from client j with $c_j^t = E^t m_j$. *FedFS* uses importance sampling for client selection to mitigate the effects introduced by this difference in client capabilities. We define the work contribution w_k of client k as the ratio between the actual work done during previous rounds $c_k = \sum_{r=1}^t c_k^r$ and the maximum work possible $\hat{c}_k = \sum_{r=1}^t E^r m_k$. Clients which have never been selected before (and hence have no contribution history) have $w_k = 0$. We then sample clients on the selection probability $1 - w_k + \epsilon$ (normalized over all $k \in \{1, \dots, K\}$), with ϵ being the minimum client selection probability. ϵ is an important hyper-parameter that prevents clients with $c_k^t = E^t m_k$ to be excluded from future rounds. Basing the client selection probability on a clients' previous contributions (w_k) allows clients which had low contributions in previous rounds to be selected more frequently, and hence contribute additional updates to the global model. Synchronous *FedAvg* is a special case of *FedFS*: if all clients are able to compute $c_k^t = E^t m_k$ every round, then there will be

Algorithm 1: FedFS

```

begin Server  $T, C, K, \epsilon, r_f, r_s, \Delta_{max}, E, B$ ,
    initialise  $\theta_0$ 
    for round  $t \leftarrow 0, \dots, T - 1$  do
         $j \leftarrow \max(\lfloor C \cdot K \rfloor, 1)$ 
         $\mathcal{S}_t \leftarrow$  (sample  $j$  distinct indices from  $\{1, \dots, K\}$ 
            with  $1 - w_k + \epsilon$ )
        if fast round ( $r_f, r_s$ ) then
             $\Delta_t = \Delta^f$ 
        else
             $\Delta_t = \Delta^s$ 
        end
        for  $k \in \mathcal{S}_t$  do in parallel
             $\theta_{t+1}^k, c_k, m_k \leftarrow \text{ClientTraining}(k, \Delta_t, \theta_t, E, B, \Delta_t)$ 
        end
         $c_r \leftarrow \sum_{k \in \mathcal{S}_t} c_k$ 
         $\theta_{t+1} \leftarrow \sum_{k \in \mathcal{S}_t} \frac{c_k}{c_r} \theta_{t+1}^k$ 
    end
end
    
```

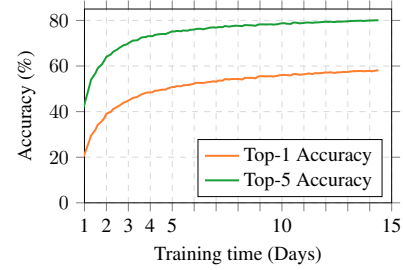


Figure 10. Training time reported in days and accuracies (Top-1 and Top-5) for an ImageNet federated training with Flower.

no difference in w_k and *FedFS* samples amongst all clients with a uniform client selection probability of $\frac{1}{K}$.

Alternating timeout. Gradual failure for clients which are not able to compute $E^t \frac{m_k}{B}$ gradient updates within Δ_t and client selection based on previous contributions allow *FedFS* to use more aggressive values for Δ . One strategy is to use an alternating schedule for Δ in which we perform r_f “fast” rounds with small Δ^f and r_s “slow” rounds with larger Δ^s . This allows *FedFS* to be configured for either improved convergence in terms of wall-clock time or better overall performance (e.g., in terms for classification accuracy).

FedFS algorithm. The full *FedFS* algorithm is given in Algorithm 1.

A.3 Scaling FedAvg to ImageNet-scale datasets

We now demonstrate that Flower can not only scale to a large number of clients, but it can also support training of FL models on web-scale workloads such as ImageNet. To the best of our knowledge, this is the first-ever attempt at training ImageNet in a FL setting.

Experiment Setup. We use the ILSVRC-2012 ImageNet

partitioning (Russakovsky et al., 2015) that contains 1.2M pictures for training and a subset composed of 50K images for testing. We train a ResNet-18 model on this dataset in a federated setting with 50 clients equipped with four physical CPU cores. To this end, we partition the ImageNet training set into 50 IID partitions and distribute them on each client. During training, we also consider a simple image augmentation scheme based on random horizontal flipping and cropping.

Results. Figure 10 shows the results on the test set of ImageNet obtained by training a ResNet-18 model. It is worth to mention that based on 50 clients and 3 local epochs, the training lasted for about 15 days demonstrating Flower’s potential to run long-term and realistic experiments.

We measured top-1 and top-5 accuracies of 59.1% and 80.4% respectively obtained with FL compared to 63% and 84% for centralised training. First, it is clear from Figure 10 that FL accuracies could have increased a bit further at the cost of a longer training time, certainly reducing the gap with centralised training. Then, the ResNet-18 architecture relies heavily on batch-normalisation, and it is unclear how the internal statistics of this technique behave in the context of FL, potentially harming the final results. As expected, the scalability of Flower helps with raising and investing new issues related to federated learning.

For such long-term experiments, one major risk is that client devices may go offline during training, thereby nullifying the training progress. Flower’s built-in support for keeping the model states on the server and resuming the federated training from the last saved state in the case of failures came handy for this experiment.

A.4 Datasets and Network Architectures

We use the following datasets and network architectures for our experiments.

CIFAR-10 consists of 60,000 images from 10 different object classes. The images are 32 x 32 pixels in size and in RGB format. We use the training and test splits provided by the dataset authors — 50,000 images are used as training data and remaining 10,000 images are reserved for testing.

Fashion-MNIST consists of images of fashion items (60,000 training, 10,000 test) with 10 classes such as trousers or pullovers. The images are 28 x 28 pixels in size and in grayscale format. We use a 2-layer CNN followed by 2 fully-connected layers for training a model on this dataset.

ImageNet. We use the ILSVRC-2012 ImageNet (Russakovsky et al., 2015) containing 1.2M images for training and 50K images for testing. A ResNet-18 model is used for federated training this dataset.